基于 LSTM 的宋诗生成

姓名：李伟龙 学号：SA25214030

实验目标

本实验使用宋诗数据集训练字符级 LSTM 语言模型，实现：

1. 古诗数据读取
2. 七言绝句筛选
3. 模型训练
4. 基于起始词生成古诗

模型以“明月”为起始词生成固定格式古诗。

In [1]:
```
import json
import re
import random
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn

from torch.utils.data import Dataset, DataLoader

seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(device)
```

```
cuda
```

数据读取

实验使用 `poet.song.40000.json` 到 `poet.song.43000.json` 中的宋诗数据。

In [2]:
```
data_dir = Path("data")

json_files = sorted(data_dir.glob("poet.song.*.json"))

data = []

for file in json_files:
    with open(file, "r", encoding="utf-8") as f:
        part = json.load(f)
        data.extend(part)
```

```
        print(f"{file.name}: {len(part)}")

print("总数据量:", len(data))
print(data[0])

all_poems = []

for item in data:
    paragraphs = item.get("paragraphs", [])
    all_poems.append(paragraphs)

print("诗歌数量:", len(all_poems))
print(all_poems[0])
```

```
poet.song.40000.json: 1000
poet.song.41000.json: 1000
poet.song.42000.json: 1000
poet.song.43000.json: 1000
总数据量: 4000
{'author': '釋義青', 'paragraphs': ['於道無所證，方通萬法路。', '或明或暗行，不慎亦不護。', '月來松色寒，雲去青山露。', '今古天台橋，幾人能得度。'], 'title': '第九十四多子塔前頌', 'id': '3a1eba1b-d886-417b-9f4e-c992246fb89c'}
诗歌数量: 4000
['於道無所證，方通萬法路。', '或明或暗行，不慎亦不護。', '月來松色寒，雲去青山露。', '今古天台橋，幾人能得度。']
```

数据预处理

为了生成固定格式古诗，本实验筛选每句七字的七言绝句。

In [3]:

```
def clean_line(line):
    line = line.strip()

    line = re.sub(
        r"[，。！？、；：,.!?;:\s]",
        "",
        line
    )

    return line


def is_qiyan_jueju(paragraphs):
    # 数据中七言绝句通常是 2 个 paragraph，每个 paragraph 包含两句七言
    if len(paragraphs) != 2:
        return False

    full_text = "".join(paragraphs)

    for line in paragraphs:
        line = clean_line(line)

        # 每个 paragraph 去掉标点后应为 14 字
        if len(line) != 14:
            return False

    return True
```

```
qiyan_poems = []

for paragraphs in all_poems:

    if is_qiyan_jueju(paragraphs):

        poem = ""

        for line in paragraphs:
            poem += clean_line(line)

        qiyan_poems.append(poem)

print("七言绝句数量:", len(qiyan_poems))

print(qiyan_poems[:5])

START_TOKEN = "<"
END_TOKEN = ">"

poems = []

for poem in qiyan_poems:
    poems.append(
        START_TOKEN + poem + END_TOKEN
    )

print(poems[0])
```

```
七言绝句数量: 906
['輕輕人問玄中旨便吐肝腸說與他木人暗皺雙眉處石女多言爭奈何', '圓缺曾伸問老翁石龜銜子引清風昨朝木馬潭中過踏出金烏半夜紅', '有無今古兩重關正眼禪人過者難欲通大道長安路莫聽崑崙敘往還', '雲淡風輕近午天望花隨柳過前川旁人不識予心樂將謂偷閑學少年', '吏身拘絆同疏屬俗眼塵昏甚瞽矇辜負終南好泉石一年一度到山中']
<輕輕人問玄中旨便吐肝腸說與他木人暗皺雙眉處石女多言爭奈何>
```

字符词表构建

神经网络无法直接处理汉字，因此需要：字符 → 数字的映射。

In [4]:

```
all_text = "".join(poems)

chars = sorted(list(set(all_text)))

char2idx = {
    char: idx
    for idx, char in enumerate(chars)
}

idx2char = {
    idx: char
    for char, idx in char2idx.items()
}

vocab_size = len(chars)

print("词表大小:", vocab_size)

print(chars[:20])
```

```
text = "明月"

ids = [char2idx[c] for c in text]

print(ids)

recover_text = "".join(
    [idx2char[i] for i in ids]
)

print(recover_text)
```

```
词表大小： 2833
['<', '>', '□', '瘖', '一', '丁', '七', '丈', '三', '上', '下', '不', '且', '丕', '世', '丘', '並', '中', '丹', '主']
[1032, 1080]
明月
```

Dataset 构建

训练目标：输入前面的字符，预测下一个字符。

In [5]:
```
class PoetryDataset(Dataset):

    def __init__(self, poems, char2idx):

        self.samples = []

        for poem in poems:

            ids = [
                char2idx[c]
                for c in poem
            ]

            x = ids[:-1]
            y = ids[1:]

            self.samples.append((x, y))

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):

        x, y = self.samples[idx]

        return (
            torch.tensor(x, dtype=torch.long),
            torch.tensor(y, dtype=torch.long)
        )

dataset = PoetryDataset(poems,char2idx)

dataloader = DataLoader(dataset,batch_size=64,shuffle=True)

x, y = next(iter(dataloader))
```

```
print(x.shape)
print(y.shape)
```

```
torch.Size([64, 29])
torch.Size([64, 29])
```

LSTM 模型

本实验采用 LSTM（Long Short-Term Memory）作为序列建模网络。LSTM 属于循环神经网络（RNN）的一种改进结构，能够有效缓解：长序列梯度消失和长期依赖难以学习等问题。

模型结构如下：

```
Token ID
→ Embedding
→ LSTM
→ Linear
→ Softmax
```

In [6]:
```
class PoetryLSTM(nn.Module):

    def __init__(
        self,
        vocab_size,
        embed_dim=128,
        hidden_dim=256,
        num_layers=2
    ):

        super().__init__()

        self.embedding = nn.Embedding(
            vocab_size,
            embed_dim
        )

        self.lstm = nn.LSTM(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=0.2
        )

        self.fc = nn.Linear(
            hidden_dim,
            vocab_size
        )

    def forward(self, x, hidden=None):

        x = self.embedding(x)

        output, hidden = self.lstm(
            x,
            hidden
        )
```

```
        logits = self.fc(output)

        return logits, hidden

model = PoetryLSTM(vocab_size)

model = model.to(device)

criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)

print(model)
```

```
PoetryLSTM(
  (embedding): Embedding(2833, 128)
  (lstm): LSTM(128, 256, num_layers=2, batch_first=True, dropout=0.2)
  (fc): Linear(in_features=256, out_features=2833, bias=True)
)
```

模型训练

本实验采用交叉熵损失函数（Cross Entropy Loss）进行训练。训练目标是最小化预测字符与真实字符之间的误差。优化器采用 Adam， 在训练过程中：输入字符序列；LSTM 输出每个位置的预测；与真实下一个字符计算交叉熵；反向传播更新参数； 随着训练进行，模型逐渐学习古诗中的字符组合规律与语言风格。 训练过程中记录每个 epoch 的平均 loss，并绘制 Loss 曲线观察模型收敛情况。

In [7]:
```
num_epochs = 300

train_losses = []

for epoch in range(num_epochs):

    model.train()

    total_loss = 0

    for x, y in dataloader:

        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()

        logits, _ = model(x)

        loss = criterion(
            logits.reshape(-1, vocab_size),
            y.reshape(-1)
        )

        loss.backward()
```

```
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)

    train_losses.append(avg_loss)

    print(
        f"Epoch [{epoch+1}/{num_epochs}] "
        f"Loss: {avg_loss:.4f}"
    )
plt.figure(figsize=(8, 5), dpi=150)

plt.plot(
    range(1, len(train_losses) + 1),
    train_losses,
    linewidth=2,
    label="Train Loss"
)

plt.xlabel("Epoch", fontsize=12)
plt.ylabel("Cross Entropy Loss", fontsize=12)
plt.title("Training Loss Curve", fontsize=14)

plt.grid(True, linestyle="--", alpha=0.4)
plt.legend()
plt.tight_layout()

plt.show()
```

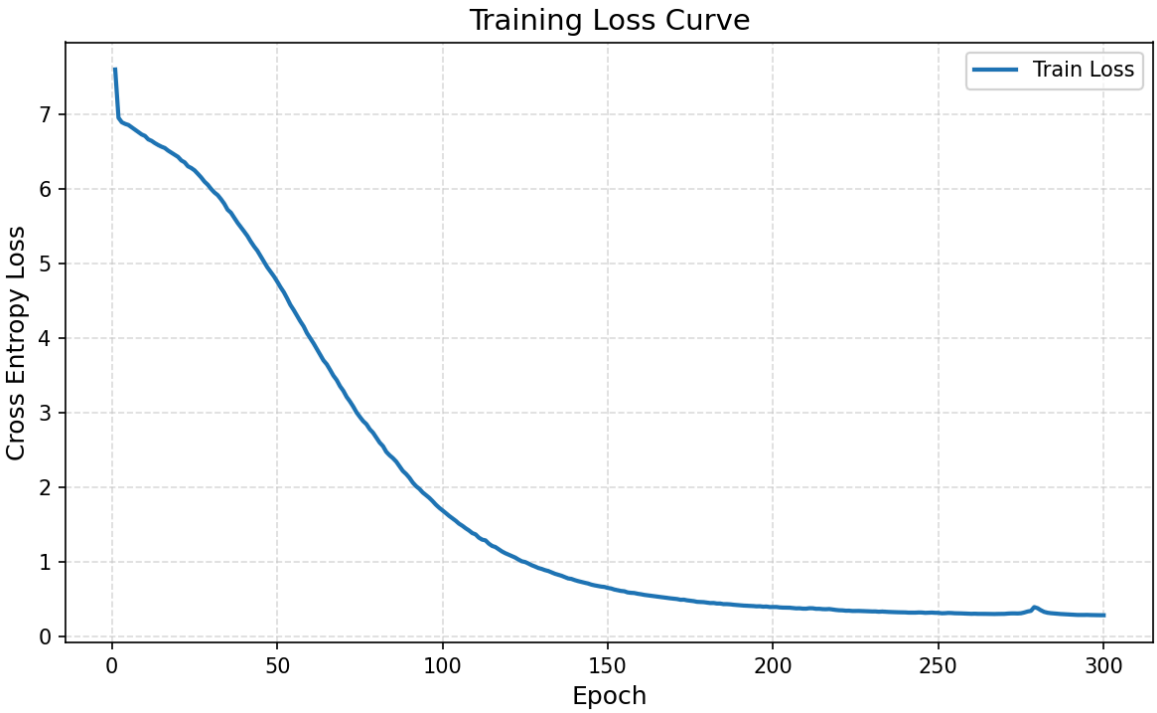
```
Epoch [1/300] Loss: 7.5963
Epoch [2/300] Loss: 6.9494
Epoch [3/300] Loss: 6.8898
Epoch [4/300] Loss: 6.8680
Epoch [5/300] Loss: 6.8533
Epoch [6/300] Loss: 6.8224
Epoch [7/300] Loss: 6.7900
Epoch [8/300] Loss: 6.7580
Epoch [9/300] Loss: 6.7266
Epoch [10/300] Loss: 6.7066
Epoch [11/300] Loss: 6.6609
Epoch [12/300] Loss: 6.6407
Epoch [13/300] Loss: 6.6104
Epoch [14/300] Loss: 6.5849
Epoch [15/300] Loss: 6.5627
Epoch [16/300] Loss: 6.5450
Epoch [17/300] Loss: 6.5105
Epoch [18/300] Loss: 6.4818
Epoch [19/300] Loss: 6.4547
Epoch [20/300] Loss: 6.4261
Epoch [21/300] Loss: 6.3786
Epoch [22/300] Loss: 6.3528
Epoch [23/300] Loss: 6.3002
Epoch [24/300] Loss: 6.2771
Epoch [25/300] Loss: 6.2457
Epoch [26/300] Loss: 6.1994
Epoch [27/300] Loss: 6.1510
Epoch [28/300] Loss: 6.0948
Epoch [29/300] Loss: 6.0530
Epoch [30/300] Loss: 5.9971
Epoch [31/300] Loss: 5.9497
Epoch [32/300] Loss: 5.9142
Epoch [33/300] Loss: 5.8596
Epoch [34/300] Loss: 5.7977
Epoch [35/300] Loss: 5.7187
Epoch [36/300] Loss: 5.6788
Epoch [37/300] Loss: 5.6106
Epoch [38/300] Loss: 5.5439
Epoch [39/300] Loss: 5.4856
Epoch [40/300] Loss: 5.4267
Epoch [41/300] Loss: 5.3660
Epoch [42/300] Loss: 5.2928
Epoch [43/300] Loss: 5.2271
Epoch [44/300] Loss: 5.1693
Epoch [45/300] Loss: 5.0954
Epoch [46/300] Loss: 5.0233
Epoch [47/300] Loss: 4.9485
Epoch [48/300] Loss: 4.8879
Epoch [49/300] Loss: 4.8264
Epoch [50/300] Loss: 4.7561
Epoch [51/300] Loss: 4.6811
Epoch [52/300] Loss: 4.6140
Epoch [53/300] Loss: 4.5317
Epoch [54/300] Loss: 4.4430
Epoch [55/300] Loss: 4.3744
Epoch [56/300] Loss: 4.2981
Epoch [57/300] Loss: 4.2234
Epoch [58/300] Loss: 4.1557
Epoch [59/300] Loss: 4.0661
Epoch [60/300] Loss: 3.9973
```

```
Epoch [61/300] Loss: 3.9287
Epoch [62/300] Loss: 3.8534
Epoch [63/300] Loss: 3.7789
Epoch [64/300] Loss: 3.7002
Epoch [65/300] Loss: 3.6478
Epoch [66/300] Loss: 3.5736
Epoch [67/300] Loss: 3.4961
Epoch [68/300] Loss: 3.4363
Epoch [69/300] Loss: 3.3550
Epoch [70/300] Loss: 3.2931
Epoch [71/300] Loss: 3.2117
Epoch [72/300] Loss: 3.1505
Epoch [73/300] Loss: 3.0819
Epoch [74/300] Loss: 3.0040
Epoch [75/300] Loss: 2.9438
Epoch [76/300] Loss: 2.8869
Epoch [77/300] Loss: 2.8452
Epoch [78/300] Loss: 2.7784
Epoch [79/300] Loss: 2.7310
Epoch [80/300] Loss: 2.6660
Epoch [81/300] Loss: 2.5990
Epoch [82/300] Loss: 2.5518
Epoch [83/300] Loss: 2.4752
Epoch [84/300] Loss: 2.4290
Epoch [85/300] Loss: 2.3912
Epoch [86/300] Loss: 2.3451
Epoch [87/300] Loss: 2.2827
Epoch [88/300] Loss: 2.2196
Epoch [89/300] Loss: 2.1783
Epoch [90/300] Loss: 2.1276
Epoch [91/300] Loss: 2.0641
Epoch [92/300] Loss: 2.0154
Epoch [93/300] Loss: 1.9799
Epoch [94/300] Loss: 1.9319
Epoch [95/300] Loss: 1.8951
Epoch [96/300] Loss: 1.8594
Epoch [97/300] Loss: 1.8154
Epoch [98/300] Loss: 1.7674
Epoch [99/300] Loss: 1.7265
Epoch [100/300] Loss: 1.6914
Epoch [101/300] Loss: 1.6550
Epoch [102/300] Loss: 1.6176
Epoch [103/300] Loss: 1.5855
Epoch [104/300] Loss: 1.5518
Epoch [105/300] Loss: 1.5131
Epoch [106/300] Loss: 1.4850
Epoch [107/300] Loss: 1.4516
Epoch [108/300] Loss: 1.4226
Epoch [109/300] Loss: 1.3874
Epoch [110/300] Loss: 1.3695
Epoch [111/300] Loss: 1.3259
Epoch [112/300] Loss: 1.2995
Epoch [113/300] Loss: 1.2895
Epoch [114/300] Loss: 1.2463
Epoch [115/300] Loss: 1.2153
Epoch [116/300] Loss: 1.2000
Epoch [117/300] Loss: 1.1710
Epoch [118/300] Loss: 1.1408
Epoch [119/300] Loss: 1.1176
Epoch [120/300] Loss: 1.0984
```

```
Epoch [121/300] Loss: 1.0806
Epoch [122/300] Loss: 1.0590
Epoch [123/300] Loss: 1.0321
Epoch [124/300] Loss: 1.0091
Epoch [125/300] Loss: 0.9993
Epoch [126/300] Loss: 0.9790
Epoch [127/300] Loss: 0.9564
Epoch [128/300] Loss: 0.9391
Epoch [129/300] Loss: 0.9193
Epoch [130/300] Loss: 0.9071
Epoch [131/300] Loss: 0.8909
Epoch [132/300] Loss: 0.8792
Epoch [133/300] Loss: 0.8611
Epoch [134/300] Loss: 0.8425
Epoch [135/300] Loss: 0.8293
Epoch [136/300] Loss: 0.8156
Epoch [137/300] Loss: 0.7976
Epoch [138/300] Loss: 0.7799
Epoch [139/300] Loss: 0.7731
Epoch [140/300] Loss: 0.7573
Epoch [141/300] Loss: 0.7441
Epoch [142/300] Loss: 0.7324
Epoch [143/300] Loss: 0.7221
Epoch [144/300] Loss: 0.7121
Epoch [145/300] Loss: 0.6972
Epoch [146/300] Loss: 0.6869
Epoch [147/300] Loss: 0.6790
Epoch [148/300] Loss: 0.6705
Epoch [149/300] Loss: 0.6657
Epoch [150/300] Loss: 0.6525
Epoch [151/300] Loss: 0.6450
Epoch [152/300] Loss: 0.6308
Epoch [153/300] Loss: 0.6222
Epoch [154/300] Loss: 0.6123
Epoch [155/300] Loss: 0.6078
Epoch [156/300] Loss: 0.5925
Epoch [157/300] Loss: 0.5872
Epoch [158/300] Loss: 0.5845
Epoch [159/300] Loss: 0.5748
Epoch [160/300] Loss: 0.5682
Epoch [161/300] Loss: 0.5595
Epoch [162/300] Loss: 0.5536
Epoch [163/300] Loss: 0.5467
Epoch [164/300] Loss: 0.5404
Epoch [165/300] Loss: 0.5345
Epoch [166/300] Loss: 0.5305
Epoch [167/300] Loss: 0.5243
Epoch [168/300] Loss: 0.5170
Epoch [169/300] Loss: 0.5130
Epoch [170/300] Loss: 0.5086
Epoch [171/300] Loss: 0.5035
Epoch [172/300] Loss: 0.4939
Epoch [173/300] Loss: 0.4954
Epoch [174/300] Loss: 0.4864
Epoch [175/300] Loss: 0.4810
Epoch [176/300] Loss: 0.4756
Epoch [177/300] Loss: 0.4665
Epoch [178/300] Loss: 0.4639
Epoch [179/300] Loss: 0.4620
Epoch [180/300] Loss: 0.4566
```

```
Epoch [181/300] Loss: 0.4501
Epoch [182/300] Loss: 0.4510
Epoch [183/300] Loss: 0.4438
Epoch [184/300] Loss: 0.4436
Epoch [185/300] Loss: 0.4355
Epoch [186/300] Loss: 0.4358
Epoch [187/300] Loss: 0.4328
Epoch [188/300] Loss: 0.4276
Epoch [189/300] Loss: 0.4238
Epoch [190/300] Loss: 0.4191
Epoch [191/300] Loss: 0.4163
Epoch [192/300] Loss: 0.4144
Epoch [193/300] Loss: 0.4119
Epoch [194/300] Loss: 0.4096
Epoch [195/300] Loss: 0.4057
Epoch [196/300] Loss: 0.4070
Epoch [197/300] Loss: 0.4015
Epoch [198/300] Loss: 0.4032
Epoch [199/300] Loss: 0.3967
Epoch [200/300] Loss: 0.3971
Epoch [201/300] Loss: 0.3974
Epoch [202/300] Loss: 0.3916
Epoch [203/300] Loss: 0.3891
Epoch [204/300] Loss: 0.3885
Epoch [205/300] Loss: 0.3876
Epoch [206/300] Loss: 0.3828
Epoch [207/300] Loss: 0.3788
Epoch [208/300] Loss: 0.3799
Epoch [209/300] Loss: 0.3751
Epoch [210/300] Loss: 0.3746
Epoch [211/300] Loss: 0.3808
Epoch [212/300] Loss: 0.3797
Epoch [213/300] Loss: 0.3727
Epoch [214/300] Loss: 0.3729
Epoch [215/300] Loss: 0.3687
Epoch [216/300] Loss: 0.3674
Epoch [217/300] Loss: 0.3699
Epoch [218/300] Loss: 0.3631
Epoch [219/300] Loss: 0.3576
Epoch [220/300] Loss: 0.3533
Epoch [221/300] Loss: 0.3520
Epoch [222/300] Loss: 0.3475
Epoch [223/300] Loss: 0.3478
Epoch [224/300] Loss: 0.3439
Epoch [225/300] Loss: 0.3439
Epoch [226/300] Loss: 0.3444
Epoch [227/300] Loss: 0.3424
Epoch [228/300] Loss: 0.3416
Epoch [229/300] Loss: 0.3391
Epoch [230/300] Loss: 0.3375
Epoch [231/300] Loss: 0.3376
Epoch [232/300] Loss: 0.3335
Epoch [233/300] Loss: 0.3363
Epoch [234/300] Loss: 0.3333
Epoch [235/300] Loss: 0.3308
Epoch [236/300] Loss: 0.3291
Epoch [237/300] Loss: 0.3288
Epoch [238/300] Loss: 0.3263
Epoch [239/300] Loss: 0.3258
Epoch [240/300] Loss: 0.3247
```

```
Epoch [241/300] Loss: 0.3217
Epoch [242/300] Loss: 0.3214
Epoch [243/300] Loss: 0.3215
Epoch [244/300] Loss: 0.3239
Epoch [245/300] Loss: 0.3233
Epoch [246/300] Loss: 0.3187
Epoch [247/300] Loss: 0.3199
Epoch [248/300] Loss: 0.3223
Epoch [249/300] Loss: 0.3193
Epoch [250/300] Loss: 0.3188
Epoch [251/300] Loss: 0.3129
Epoch [252/300] Loss: 0.3141
Epoch [253/300] Loss: 0.3178
Epoch [254/300] Loss: 0.3164
Epoch [255/300] Loss: 0.3131
Epoch [256/300] Loss: 0.3128
Epoch [257/300] Loss: 0.3113
Epoch [258/300] Loss: 0.3098
Epoch [259/300] Loss: 0.3093
Epoch [260/300] Loss: 0.3058
Epoch [261/300] Loss: 0.3072
Epoch [262/300] Loss: 0.3053
Epoch [263/300] Loss: 0.3057
Epoch [264/300] Loss: 0.3050
Epoch [265/300] Loss: 0.3043
Epoch [266/300] Loss: 0.3044
Epoch [267/300] Loss: 0.3030
Epoch [268/300] Loss: 0.3043
Epoch [269/300] Loss: 0.3040
Epoch [270/300] Loss: 0.3052
Epoch [271/300] Loss: 0.3090
Epoch [272/300] Loss: 0.3112
Epoch [273/300] Loss: 0.3114
Epoch [274/300] Loss: 0.3101
Epoch [275/300] Loss: 0.3132
Epoch [276/300] Loss: 0.3236
Epoch [277/300] Loss: 0.3368
Epoch [278/300] Loss: 0.3452
Epoch [279/300] Loss: 0.3957
Epoch [280/300] Loss: 0.3812
Epoch [281/300] Loss: 0.3535
Epoch [282/300] Loss: 0.3318
Epoch [283/300] Loss: 0.3203
Epoch [284/300] Loss: 0.3143
Epoch [285/300] Loss: 0.3105
Epoch [286/300] Loss: 0.3068
Epoch [287/300] Loss: 0.3031
Epoch [288/300] Loss: 0.3007
Epoch [289/300] Loss: 0.2985
Epoch [290/300] Loss: 0.2950
Epoch [291/300] Loss: 0.2926
Epoch [292/300] Loss: 0.2909
Epoch [293/300] Loss: 0.2903
Epoch [294/300] Loss: 0.2905
Epoch [295/300] Loss: 0.2914
Epoch [296/300] Loss: 0.2896
Epoch [297/300] Loss: 0.2893
Epoch [298/300] Loss: 0.2879
Epoch [299/300] Loss: 0.2866
Epoch [300/300] Loss: 0.2865
```



古诗生成

模型训练完成后，实验使用自回归方式生成古诗。

生成流程如下：

1. 输入起始词“明月”；
2. 模型预测下一个字符概率；
3. 根据概率分布随机采样；
4. 将生成字符拼接回输入；
5. 继续预测下一个字符；
6. 重复直到生成 28 个字符。

本实验采用Temperature Sampling控制生成随机性。其中：temperature较小，生成更稳定，较大，生成更多样。

In [8]:
```
def generate_poem(
    model,
    start_text="明月",
    max_len=28,
    temperature=0.8
):

    model.eval()

    result = start_text

    # 起始输入
    input_ids = [
        char2idx[c]
        for c in START_TOKEN + start_text
        if c in char2idx
    ]
```

```
    input_tensor = torch.tensor(
        [input_ids],
        dtype=torch.long
    ).to(device)

    hidden = None

    with torch.no_grad():

        # 先喂入起始文本
        _, hidden = model(
            input_tensor,
            hidden
        )

        current_id = input_tensor[:, -1:]

        while len(result) < max_len:

            logits, hidden = model(
                current_id,
                hidden
            )

            # 取最后一个位置
            logits = logits[:, -1, :]

            # temperature 控制随机性
            logits = logits / temperature

            # 禁止提前结束
            logits[:, char2idx[END_TOKEN]] = -1e9

            probs = torch.softmax(
                logits,
                dim=-1
            )

            # 随机采样
            next_id = torch.multinomial(
                probs,
                num_samples=1
            )

            next_char = idx2char[
                next_id.item()
            ]

            result += next_char

            current_id = next_id

    return result

def format_qiyan(text):
    text = text[:28]

    lines = [
        text[:7],
```

```
        text[7:14],
        text[14:21],
        text[21:28]
    ]

    return (
        lines[0] + ", \n" +
        lines[1] + "。\n" +
        lines[2] + ", \n" +
        lines[3] + "。"
    )
for i in range(5):

    poem = generate_poem(
        model,
        start_text="明月",
        max_len=28,
        temperature=0.8
    )

    print(f"生成结果 {i+1}")
    print(format_qiyan(poem))
    print()
```

生成结果 1
明月明何郎侯登，
陟崎嶇力不可是。
終日木馬裹清斜，
散壓雙雙回過空。

生成结果 2
明月明朝往多茅，
上依拂金微也得。
飛行細草已落魚，
樽未愧始重悲陽。

生成结果 3
明月明和夢清樽，
賊條白岡翠就降。
有六但似君起起，
否入五心蟹骨追。

生成结果 4
明月明隱吾家馬，
晚清驚論舊狄海。
春莫道長安入近，
來和騎萬挑存塊。

生成结果 5
明月明吾事雜梅，
把雨還見詩天只。
自期李住玉堂方，
後展老師蜀世緣。